SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026
Email: umhackathon@um.edu.my

1. Introduction

This section introduces the proposed Hospital AI Decision Intelligence System (HaiDIS), a strategic solution designed to address inefficiencies in hospital resource management identified in the problem statement. The system focuses on improving operational efficiency, reducing patient waiting times, and supporting data-driven decision-making in healthcare environments.

1.1. Purpose

This system analysis document outlines the technical scope, system architecture, and design decisions behind the development of the Hospital AI Decision Intelligence System. It serves as a guide to understanding how the system components interact to deliver real-time insights, AI-driven recommendations, and simulation capabilities.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
   a. The system is designed based on a modular, full-stack web application architecture integrated with AI services. It is divided into multiple layers to ensure scalability, maintainability, and separation of concerns.
   b. **Frontend layer**
      HTML templates and static assets located in the templates/ and static/ directories, including the main dashboard and simulator interface (simulator.html).
   c. **Backend layer**
      Built using Flask (app.py) and serves as the core processing unit of the system. It handles routing, business logic, and communication between system components. Supporting modules such as analytics.py and costvsefficiency.py manage data processing and computational logic, while config.py manages system configurations.
   d. **AI service layer**
      Implemented in ai_service.py. Responsible for integrating external AI models or APIs to generate intelligent recommendations based on hospital data inputs.
   e. **Data layer**
      Structured datasets stored in the data/ directory and Dataset.zip, which are used for simulation, analysis, and model input processing.
   f. **Deployment and execution configuration**
      Defined using requirements.txt and Procfile, ensuring the system can be deployed in a web environment.

2. **Data Flows:**
   a. The system follows a structured data flow pipeline beginning from data ingestion through to output generation, with two distinct flows, one for the dashboard and one for the simulator page.
   b. On the dashboard (index.html), there is no direct user input. Instead, app.js automatically fetches live data from the backend via four API endpoints (/api/context, /api/recommendations, /api/cost-efficiency, and /api/simulate-dashboard) and renders the results in real time without any user interaction required.

© UMHackathon 2026

c. On the simulator page (simulator.html), the user provides numeric inputs (additional doctors, nurses, and beds). These inputs are transmitted to the backend via two API calls: /api/simulate which runs the what-if calculation, and /api/recommendation which sends the simulation result to GLM for AI-generated analysis.
d. Within the backend, two parallel processing paths operate independently. The first path involves analytics.py, which reads and aggregates live pediatric metrics from the hospital management database, updated every 5 minutes with randomised data to simulate real hospital conditions into a compact JSON payload. This payload is passed to ai_service.py, which calls the ILMU GLM-5.1 API and returns three prioritised recommendations to the dashboard. The second path involves costvsefficiency.py, which performs pure mathematical calculations on the current hospital state to produce cost and waiting time metrics for the live simulation on the dashboard. For the simulator page, the simulate() function handles the what-if calculation independently before ask_ai_for_simulation() sends the result to GLM for narrative analysis.
e. The processed outputs from both paths are returned to the frontend, where they are displayed as real-time zone status, AI-generated recommendations, cost vs efficiency comparisons, live simulation metrics, and what-if analysis results across the dashboard and simulator pages.

3. **Model Process:**
   a. The system follows an end-to-end operational workflow designed to support hospital decision-making.
   b. Initially, hospital operational data such as patient count, staff availability, and bed capacity is collected and loaded into the system. This data is processed through backend modules to generate structured insights.
   c. The AI recommendation engine (ai_service.py) is then triggered, where processed hospital data is sent to an AI model or API to generate context-aware recommendations.
   d. The What-If Simulator allows users to modify operational variables such as staff and bed allocation. The system processes these inputs to generate predictive outcomes based on defined logic and constraints.
   e. The Cost vs Efficiency module evaluates trade-offs between resource allocation and operational performance.
   f. The final outputs, including AI recommendations, simulation results, and analytical insights, are presented on the dashboard for decision-making purposes.

4. **Role of Reference:**
   a. This document serves as a comprehensive technical reference for the Hospital AI Decision Intelligence System (HaiDIS).
   b. It provides developers with a clear understanding of the system architecture, module responsibilities, and data flow structure, enabling efficient development, integration, and maintenance of the system.
   c. For testers, this document acts as a guideline for validating system functionality, ensuring that each component operates according to defined requirements and expected outputs.

© UMHackathon 2026

d. For stakeholders, it provides a high-level overview of how the system processes hospital data and generates intelligent recommendations to support decision-making.
e. Overall, this document ensures consistency across development, testing, and system evaluation processes.

1.2. Background

Hospitals in Malaysia are facing increasing challenges such as overcrowding, staff shortages, and inefficient resource allocation. Existing systems often rely on static data and manual planning, limiting the ability to respond quickly to changing conditions. This creates a need for a smarter, AI-driven solution that can provide real-time insights and support better decision-making.

1. **Previous Version:**
   a. Decisions are often based on historical data rather than real-time conditions, making it difficult to respond quickly to sudden patient surges or staff shortages.
   b. This leads to inefficient workflows, overcrowding, underutilisation or overuse of resources, and increased pressure on healthcare workers.
   c. The lack of integrated, intelligent systems limits the ability of hospital administrators to make fast, informed, and data-driven decisions.

2. **Changes in Major Architectural Components:**
   a. Integration of AI recommendation engine for intelligent decision support.
   b. Introduction of real-time data processing and live dashboard updates.
   c. Addition of What-If Simulator for scenario-based analysis.
   d. Implementation of cost vs efficiency analysis module.
   e. Shift from static systems to a dynamic, data-driven architecture.

3. **New capabilities that are only introduced in this:**
   a. Real-time monitoring of hospital operations.
   b. Predictive and AI-driven recommendations.
   c. Interactive simulation for decision testing.
   d. Data-driven cost optimization insights.
   e. Improved resource planning and operational efficiency.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Hospital Administrators / Management	Monitor hospital performance and make strategic decisions.	Accurate insights, improved efficiency, reduced waiting time.
Healthcare Operations Managers	Manage staff, beds, and daily hospital workflow.	Real-time data, better resource allocation tools.

© UMHackathon 2026

Doctors & Nurses (Supervisory Roles)	Oversee patient flow and workload distribution.	Balanced workload, visibility of resource availability.
Development Team	Build the platform and maintain it.	Clear architecture, modular design, well-defined APIs.
QA Team	Validate system functionality and performance.	Defined requirements, full test coverage, reliable outputs.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Web-based Management Dashboard (Desktop optimized)
Architecture	Client-Server Architecture with External AI Integration

HaiDIS is structured as a client-server based system. The frontend consists of a web-based dashboard that allows hospital administrators to view current resource conditions and run simulations. This dashboard communicates with a backend service, which acts as the central controller responsible for managing data flow and coordinating communication with the external AI GLM service. Through this integration, the system transforms hospital data into real-time recommendations and predictive insights. This enables a smooth flow from data processing to actionable decision-making, supporting users in making informed and efficient operational choices.

2.1.2 LLM as Service Layer

In HaiDIS, the GLM model is implemented as a **service layer** rather than a standalone internal module. The backend (ai_service.py) sends structured hospital data, such as patient count, staff availability, and bed capacity, to the GLM API through contextual prompts. The AI then processes this input and returns dynamic, data-driven recommendations.

This separation ensures:

- Clear abstraction between system logic and AI processing
- Flexibility to update or replace the AI model without affecting core system architecture
- Scalable integration of advanced AI capabilities

The AI service layer acts as an intelligent decision-support component, transforming processed hospital data into actionable insights that enhance operational efficiency.

© UMHackathon 2026

2.1.3 Dependency Diagram

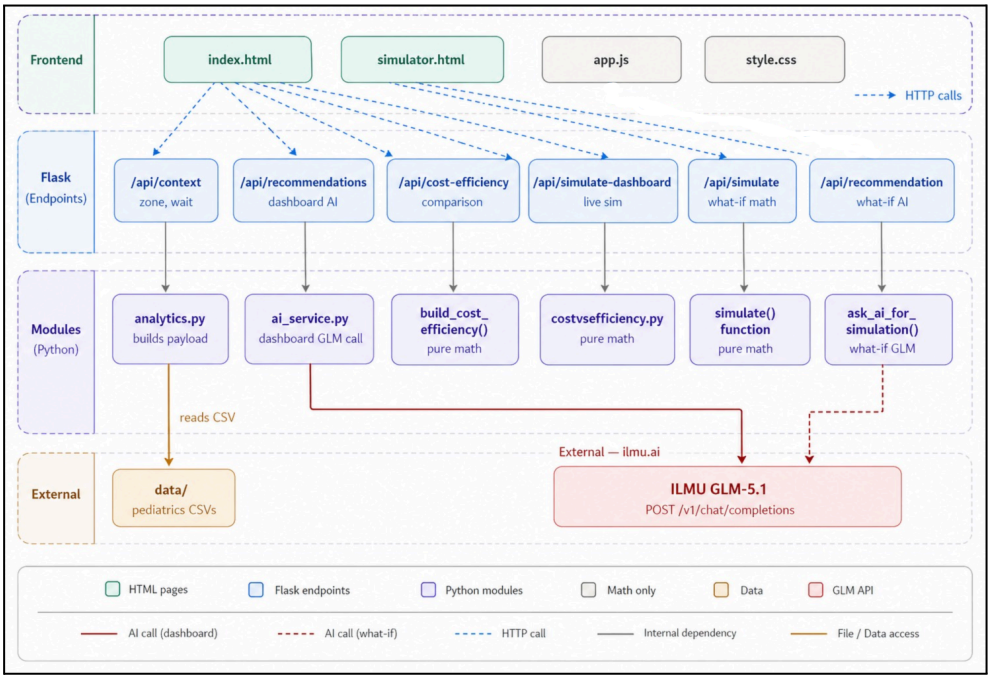


Figure 2.1.3a: Overall system architecture

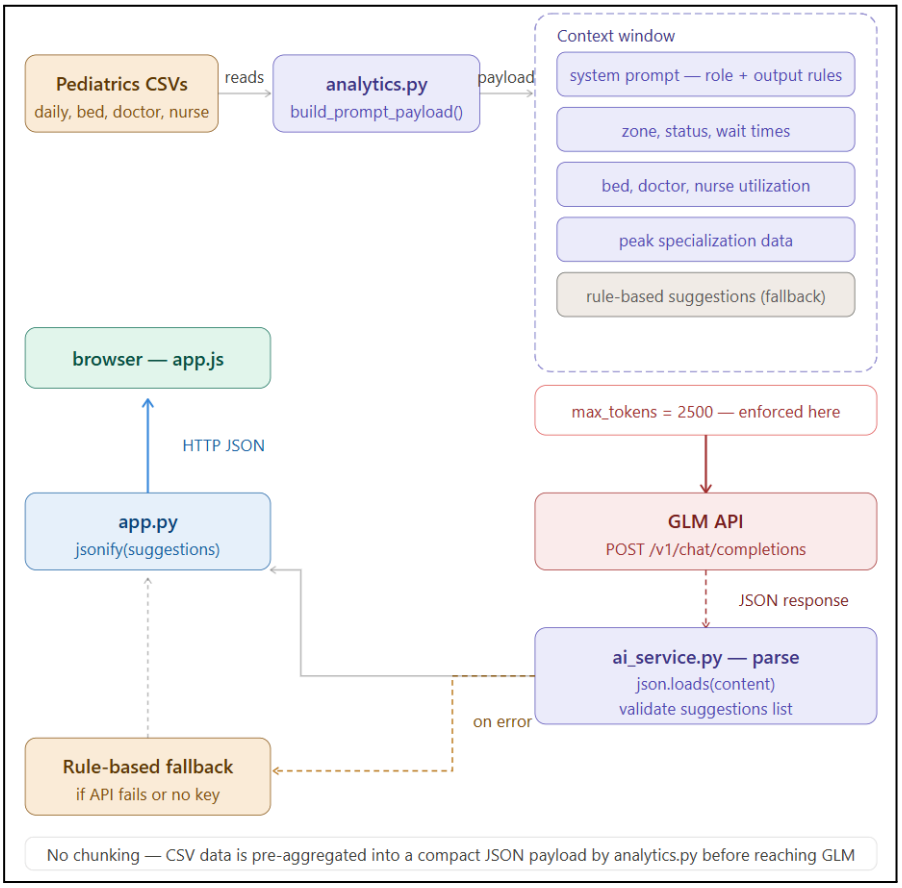


Figure 2.1.3b: GLM Prompt & Response Flow

© UMHackathon 2026

The first diagram shows the overall system architecture showing all layers and how they interact. The second diagram zooms into how the prompt is constructed and sent to GLM, and how the response is parsed back.

2.1.4. Sequence Diagram

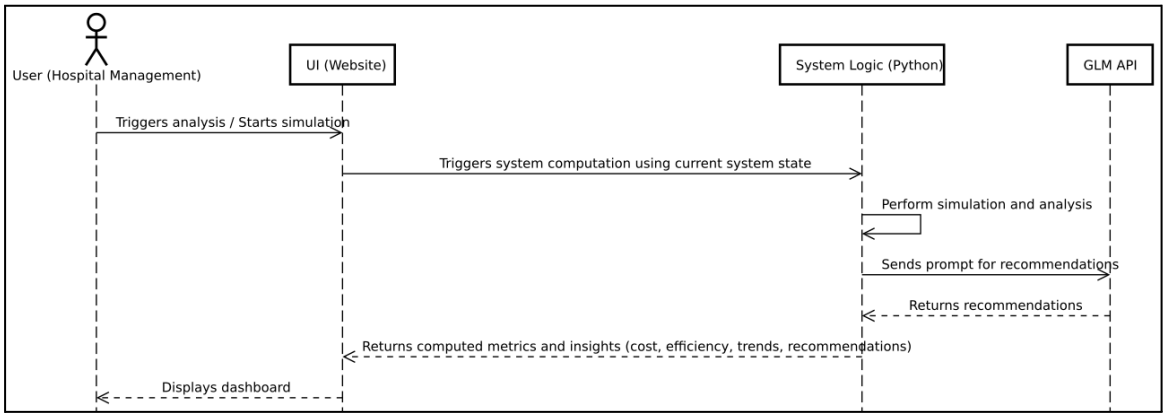


Figure 2.1.4a: Sequence diagram of AI recommendation and Cost VS Efficiency Analysis

- Step 1: The Hospital Manager initiates the process by triggering the analysis through the JavaScript-based dashboard.
- Step 2: The system's internal Python simulation logic processes the current system state by simulating changes in patient load and staff availability.
- Step 3: The system performs deterministic calculations to compute operational cost and estimated waiting time, followed by comparative analysis with previous states to evaluate efficiency changes.
- Step 4: Based on the computed data, the system generates insights using rule-based logic. The system sends the current context to the GLM API to obtain recommendation insights.
- Step 5: The system aggregates all computed metrics and insights, and returns the results to the dashboard interface, where they are displayed to support decision-making.

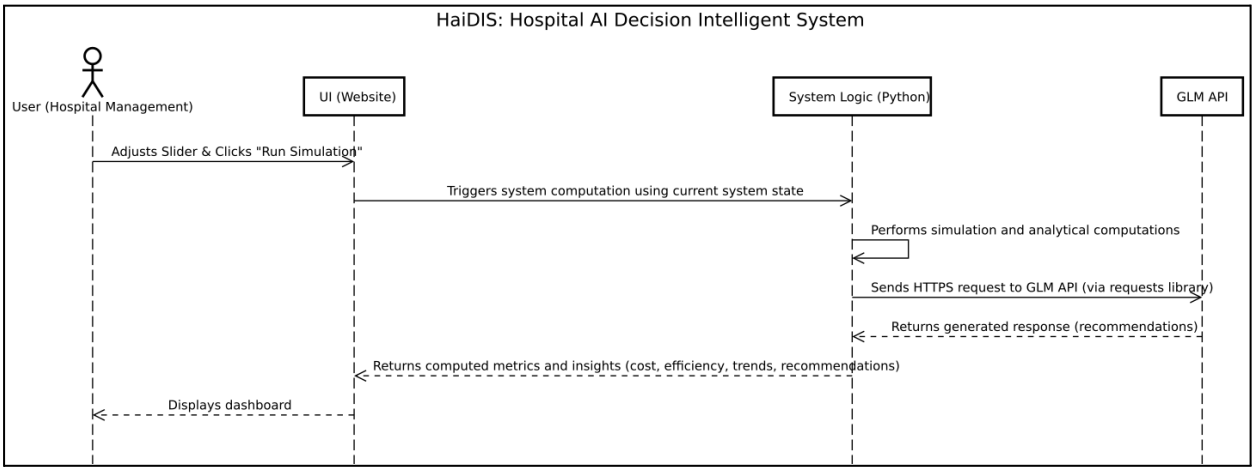


Figure 2.1.4b: Sequence Diagram of the What-If Simulator feature

- Step 1: The Hospital Manager adjusts simulation parameters (e.g., number of doctors, patients) via the dashboard.
- Step 2: The UI sends the updated parameters to the system's internal logic.
- Step 3: The Python simulation logic recalculates cost, waiting time, and efficiency metrics based on the modified inputs.
- Step 4: The system compares the new scenario with the baseline or previous state.
- Step 5: Updated results and insights are returned to the dashboard and displayed in real time.

2.2. Technological Stack

Component	Description	Technology
Frontend (Markup & Styling)	Structure and styling of the HaiDIS dashboard interface	HTML5, CSS3
Frontend (Interactivity)	Handles all API calls to Flask backend, dynamic DOM updates, and live simulation update loop every 10 seconds	Vanilla JavaScript (app.js)
Frontend (Typography)	Custom fonts used across the dashboard UI	Google Fonts (Lexend Deca, Bebas Neue)
Backend (Web Framework)	Serves HTML templates and exposes all RESTful API endpoints	Python 3, Flask
Backend (AI Client)	Communicates with ILMU GLM-5.1 via OpenAI-compatible interface to request and parse recommendations	OpenAI Python Client
Backend (Data Processing)	Reads, processes, and aggregates hospital operational data into structured payloads for GLM	Pandas
Database	External database maintained by hospital management staff, serving as the live data source for all operational metrics	Hospital Management Unit Database
AI / External API	Large language model that generates context-aware, prioritised hospital resource recommendations	ILMU GLM-5.1 (Z.AI)
Deployment (Hosting)	Hosts the full Flask application including frontend static files and backend server as a managed web service	Railway.app
Deployment (Version Control)	Repository management and source of truth, integrated with Railway for automatic redeployment on push to main	GitHub

© UMHackathon 2026

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

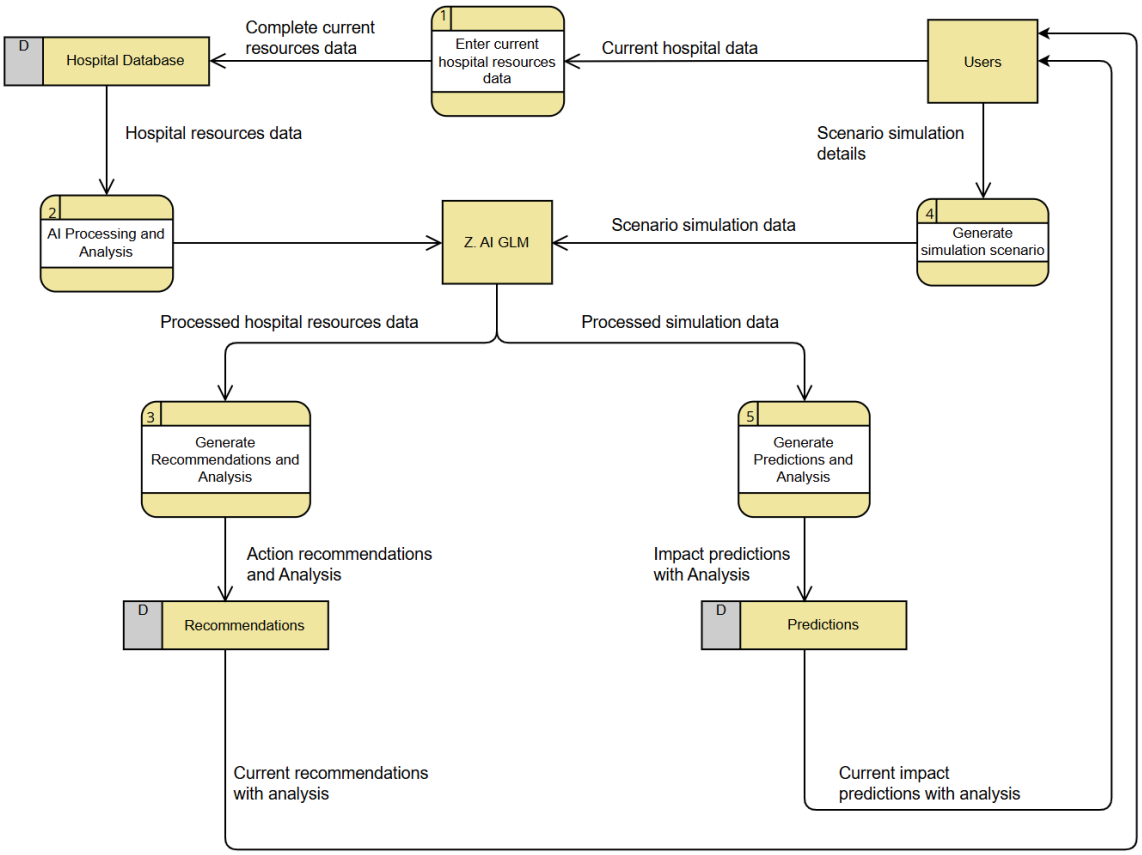


Figure 2.3.1.1: Data Flow Diagram (DFD)

2.3.2. Normalized Database Schema

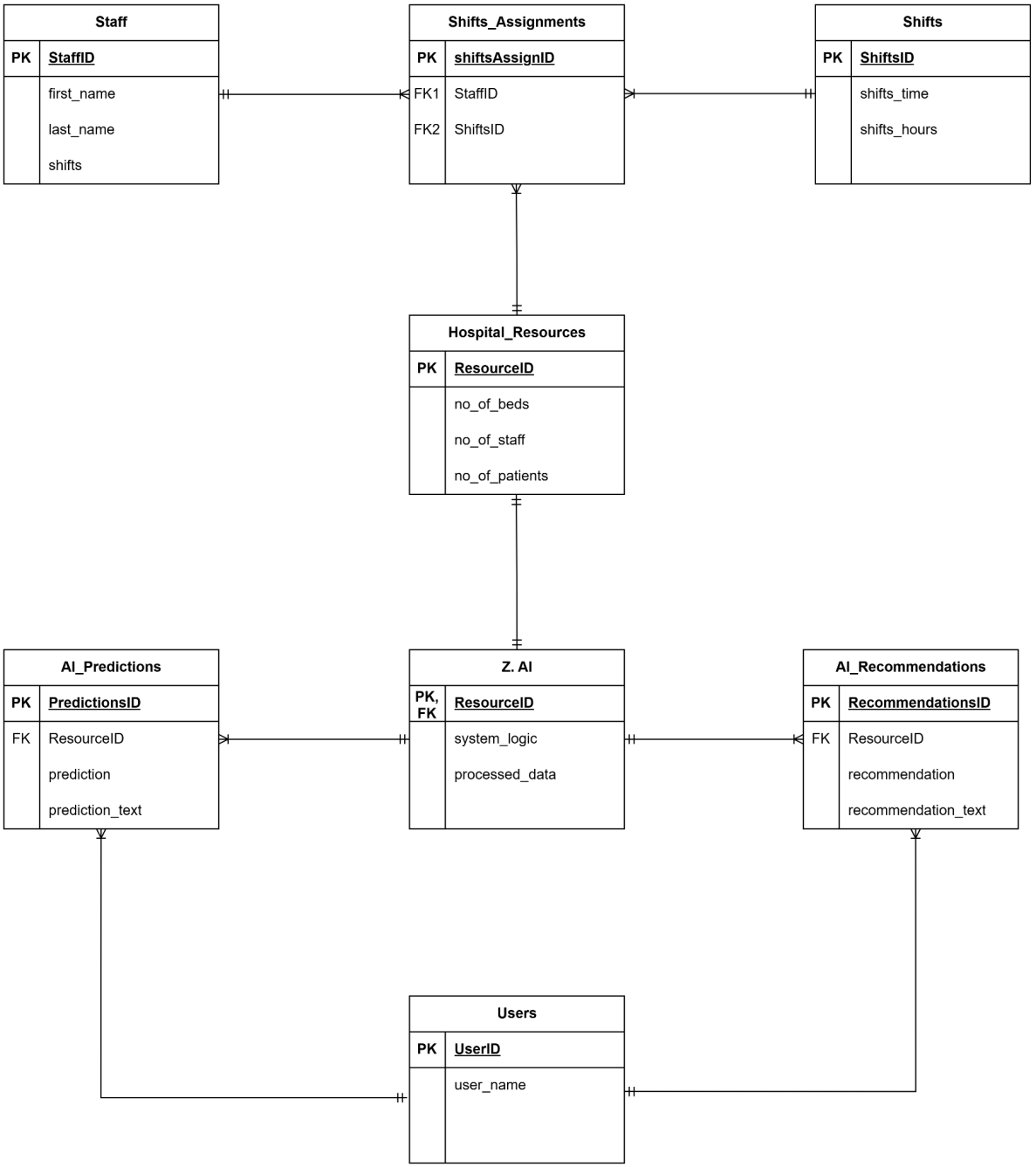


Figure 2.3.2: Normalized Entity Relationship Diagram (ERD)

3. Functional Requirements & Scope

3.1 Minimum Viable Product

No	Features	Description
1	AI Recommendation System	The system will analyze data from the hospital, such as the number of patients, staff allocated, and bed capacity. It will then generate recommendations that can improve overall healthcare service efficiency, such as assigning 3 more staff in specific room to reduce the waiting time, and reallocate beds to save costs.
2	Cost vs Efficiency Analysis	This analysis shows the trade-offs between cost and efficiency. Users will be informed of the changes in cost when resources are added/removed, and the impact it has on the efficiency which is measured by waiting time.
3	What-If Simulation	Users will be able to simulate changes in variables such as number of beds, doctors and nurses, and the system will predict and show the impact each modification has on the waiting time, operational cost, and numbers of patients served.
4	Automated Data Generation	The system will automatically generate hospital data such as number of staff, patients, and beds to simulate real-time data analysis for the system to execute its functions.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	The system must support multiple users accessing the website simultaneously without significant performance degradation.	Deploy on scalable platforms such as Vercel, and lightweight frontend design.
Reliability	The system should generate recommendations and simulation results at all times, without crashing or inconsistent output.	Implement input validation and fallback logic to handle invalid or missing data.
Maintainability	The system should be easy to update and modify, especially for UI components and logic rules	The system's code is separated into HTML, CSS, JavaScript, and Python, and modular functions are used for better readability and updates.

© UMHackathon 2026

Performance	The system response time should be fast for all user interactions, such as generating recommendations and running simulations.	Client-side processing ensures fast response time for simulations and recommendations.
Usability	The system must be easy to use and navigate for all users, including non-technical users such as hospital administrators.	Simple and clean UI design with clear labels, cards and interactive buttons.
Cost Efficiency	The system should minimize unnecessary usage of AI resources while maintaining useful outputs.	Optimize AI usage by sending only essential input data, limiting prompt size, and avoiding redundant API calls.

3.3 Out of Scope / Future Enhancements

a. Real-time Hospital Data Integration: Connect to live hospital databases for real-time analysis and recommendations.
b. Data Visualization Enhancement: Include charts and analytic dashboards for deeper insights.
c. Mobile Application Support: Future version will be available as a mobile application for better accessibility.
d. Automated Smart Alerts: Implement smart alert systems to warn users about future risks such as resource shortage.
e. User Authentication and Role Management: Users login systems based on roles to guard system's security and data privacy.
f. Integration with Scheduling Systems: Integration with hospital staff scheduling system for automated shift assignments.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Gradual Rollout & A/B Testing

A gradual (grayscale) rollout strategy is implemented to ensure system stability and minimize risks during deployment. New features such as AI Recommendations, Cost vs Efficiency Analysis, and the What-If Simulator are first released to a small subset of users (5–10% of hospital administrators or staff).

During this phase, system performance and user interactions are closely monitored, including:

- Accuracy of AI-generated recommendations
- System response time
- Impact on decision-making efficiency
- Error rates and system crashes

Then, to ensure that HaiDIS delivers the most effective strategies for hospital management, A/B testing is conducted on key system features.

For example [AI Recommendation Variants]:
- Version A: Suggests increase staff
- Version B: Suggests optimize scheduling

Performance is evaluated based on:
- Reduction in patient waiting time
- Increase in patient throughput
- Cost efficiency improvements

4.1.2. Emergency Rollback (ER) & Golden Release:

In the event of system anomalies (incorrect recommendations, system crashes, or performance degradation), an Emergency Rollback (ER) mechanism is triggered. This involves:
- Immediate suspension of the faulty feature
- Reverting the system to the last stable version

HaiDIS maintains a "Golden Version", which is a fully tested and stable release of the system.

In case of deployment failure or major bugs:
- The system automatically redeploys the Golden Version.
- All unstable updates are rolled back.
- Services continue without downtime.

This ensures:
- High system availability.
- Reliable performance in critical environments.
- Safe recovery during emergencies.

4.1.3. Priority Matrix

Impact ↓ / Urgency →	Low Urgency	Medium Urgency	High Urgency	Critical Urgency
🔴 **P1 – Critical (High Impact)** - AI recommendations produce incorrect or misleading outputs affecting decision-making - System downtime or backend failure (Flask server/API not responding) - What-If Simulator produces invalid or corrupted results	Monitor briefly before action	Immediate investigation	Rapid fix deployment	Emergency Rollback to Golden Release
🟠 **P2 – Significant (High Impact)** - API response time exceeds acceptable threshold - AI recommendations are delayed or partially unavailable - Simulation results are slow or occasionally inaccurate	Schedule fix	Prioritize fix	Hotfix deployment	Escalate to P1 if worsens
🟡 **P3 – Moderate (Medium Impact)** - Incorrect display of analytics - Minor delays in loading dashboard data - Non-critical feature malfunction	Log for backlog	Fix in next release	Prioritized patch	Escalate if affecting users widely
🟢 **P4 – Minor (Low Impact)** - UI misalignment, styling inconsistencies - Typographical errors in system messages - Minor visual glitches in charts	Ignore temporarily	Fix during maintenance	Minor patch	Rare escalation
🔵 **P5 – Low (Very Low Impact)** - User suggestions - Improvement ideas	Backlog / ignore	Consider improvement	Optional enhancement	No escalation needed

4.2. Monitoring

4.2.1. Overview

HaiDIS uses a continuous monitoring system to ensure the system runs smoothly and any issues are detected early. The monitoring focuses on system performance, data accuracy, and overall system health.

4.2.2. Trigger Conditions

The system will detect issues when:

- System Failure:
  - Server is down or API not responding
  - Trigger: Critical issue (P1)
- Slow Performance
  - Response time more than 10 seconds
  - Trigger: High issue (P2)
- Data Errors
  - Data mismatch or incorrect results (>10%)
  - Trigger: Critical issue (P1)

4.2.3. Actions Taken

Once a problem is detected:

- P1 (Critical): Immediate rollback to stable (Golden Release)
- P2 (High): Fix quickly and monitor
- P3–P5 (Lower issues): Fix later in updates

© UMHackathon 2026

4.3. Assumptions

a. The hospital management unit maintains and updates the database regularly, ensuring that the data consumed by HaiDIS reflects current hospital conditions at all times.
b. Management staff responsible for data entry are trained and consistently input accurate, complete records into the hospital management system because HaiDIS does not validate the correctness of incoming data at the source.
c. Hospital administrators and clinical staff accessing HaiDIS have stable internet connections on their devices, as the dashboard requires continuous connectivity to fetch live data and AI recommendations from the Flask backend.
d. The ILMU GLM-5.1 API remains accessible and available during operational hours. HaiDIS assumes the external API is reachable, and only falls back to rule-based suggestions when the API is explicitly unavailable or returns an unusable response.
e. The ILMU_API_KEY stored as an environment variable in the deployment environment is valid and has sufficient quota to serve recommendation requests throughout the operational period.
f. The hospital operates within a single pediatrics department context. HaiDIS is designed and tested specifically for pediatrics data and assumes the data schema from the hospital management database remains consistent with the expected column structure.
g. The Vercel deployment environment remains available and functional as the hosting platform for both the frontend static files and the Flask serverless backend.
h. Users accessing HaiDIS are authorized hospital personnel. The current version assumes no public-facing access and does not implement user authentication or role-based access control.

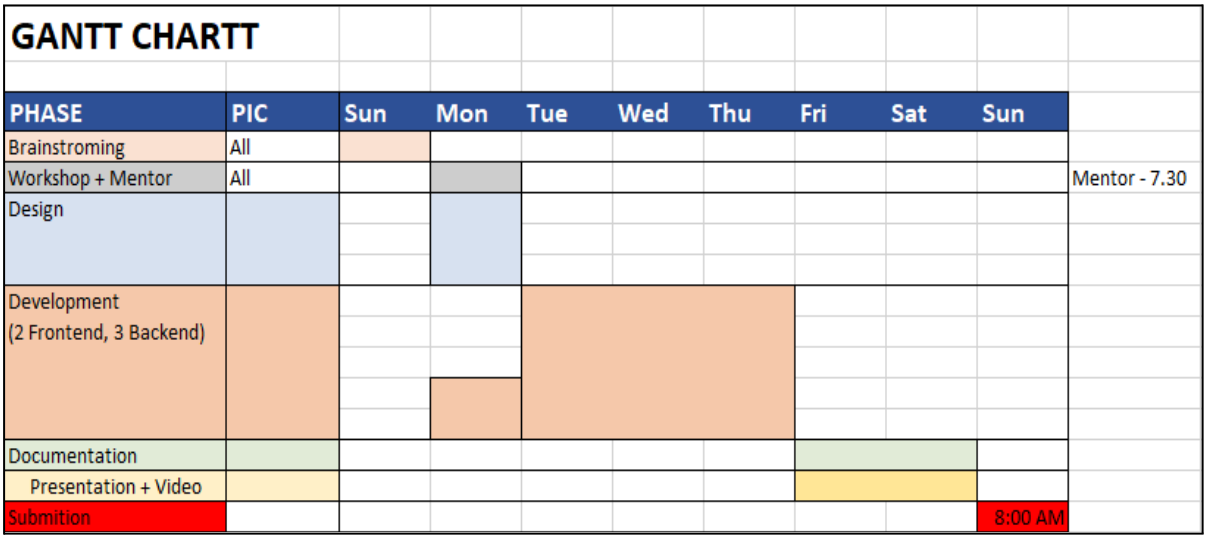
4.4. External Dependencies

Tools	Purpose	Risks
Z AI GLM API	It should be the core LLM reasoning engine that will handle the prompt inference, multi-step reasoning and generating structured output	High - in case of GLM API rate limitation, core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]

© UMHackathon 2026

5. Project Management & Team Contributions

5.1. Project Timeline



The project is structured across a condensed 10-day sprint for the Preliminary Round. Although development officially commenced on Day 3, the team utilized an Agile Crash Course approach to recover the schedule. By overlapping the System Analysis (SAD) and UI Prototyping phases, workflow was optimized to ensure all core features (AI Recommendations, Cost Analysis, and What-If Simulator) are fully documented and ready for the technical implementation phase without compromising on quality or system stability.

5.2. Team Members Role

Team member	Role	Responsibilities
Fatin Nur Liyana	Group leader	• Oversees and supervises all aspects of the project • Ensures all project milestones are met within the 10-day sprint • Conducts the final review of all deliverables • Coordinates team tasks • Resolves any "blockers" during the transition from documentation to development
Anis Atirah	Lead Backend Developer	• Sets up the FastAPI (Python) environment • Designs the API endpoints • Manages the connection to the GLM API • Develops the backend for the What-If Simulator feature
Najah Safwah	Lead Frontend Developer	• Leads the coding of the dashboard using HTML, CSS, and JavaScript • Implements the "What-If" sliders and ensures the charts update dynamically based on backend data

		• Ensures the UI correctly fetches and displays the JSON results from the Python server
Nurin Allya Insyira	Documentation Consultant	• Serves as the lead author for Document 1 (Product Requirement) and Document 2 (QA Testing) • Defines the risk mitigation strategies • Ensures all testing cases (AI output, boundary testing) are scientifically sound • Ensures all documentation follows the UMH formatting and technical requirements
Nurul Hannah	System Analyst	• Develops the mathematical backend for Cost vs Efficiency calculations • Responsible for the technical logic of the entire platform through diagrams • Translates the “Smart Hospital” problem into functional technical requirements for the developers • Designs how data moves from user input to the AI and back

5.3. Recommendations

- Performance: Using Redis Cache for frequent queries
  - Why: If multiple hospital managers ask the same "What-If" questions (e.g., "What happens if we add 5 beds?"), the system shouldn't waste money and time asking the AI again.
  - The Recommendation: Implement Redis Cache to store AI responses for common resource scenarios for 24 hours. This reduces latency and keeps the dashboard feeling "snappy."
- Reliability: Load balancing and auto-scaling
  - Why: A hospital system cannot crash during peak hours (like a sudden emergency surge).
  - The Recommendation: Deploy the FastAPI backend using AWS Elastic Load Balancing (ELB) and Auto Scaling Groups (ASG). This ensures that if traffic spikes, the system automatically creates more server "clones" to handle the load without slowing down.
- Scalability: Asynchronous task queuing (Celery)
  - Why: Sometimes the GLM API might take 3-5 seconds to think. We don't want the whole backend to "freeze" while waiting for one user's response.
  - The Recommendation: Use Celery with RabbitMQ to handle AI requests in the background. This allows the system to process hundreds of simulations at once without the server getting "stuck" on a single long-running task.

To ensure the platform is production-ready, we recommend implementing Redis Caching for repetitive AI prompts to save on token costs and AWS Auto-Scaling to maintain 99.9% uptime during hospital peak hours. Additionally, adopting Asynchronous Task Queues will prevent system bottlenecks during high-concurrency "What-If" simulations.

© UMHackathon 2026